微信开发工程师总结的20条C++性能陷阱与实用特性，读起来很过瘾

腾讯技术工程 2025年9月26日 02:36 广东 25人



作者：尚晋的思考

文章分为上下篇，上篇介绍10个C++性能陷阱，下篇介绍10个C++实用特性。

> 如果你让每个 C++工程师列出他们喜欢 C++的原因，那“掌控力”绝对是排在前几的特性。与 go、java 等垃圾回收语言的大道至简、python 等解释语言的小快灵不同，C++最大的魅力就是给予工程师对代码完全的掌控，每个 C++程序员仿佛都是人形编译器，不止要看懂代码表面的逻辑，甚至要知道每行代码对应的汇编指令。优化代码也成了 C++工程师日常必备活动，正所谓“一杯茶，一包烟，一段代码，优化一天”。在经历过无数个性能优化的日日夜夜后，笔者也总结了10中过招的性能陷阱，与诸位分享。

性能陷阱主要分为两大类：“**有成本抽象**”和“**与编译器作对**”。前者是指在使用 C++的功能/库时需要注意的隐形成本，后者则是一些 C++新手可能会写出不利于编译器优化的代码。另外本文的顺序是由基础到进阶，读者可以根据目录直接跳到自己想看的部分。

有成本抽象

C++“信徒”们常常鼓吹 C++的“零成本抽象（Zero Cost Abstraction）”。然而对于“零成本抽象”这个概念存在很多误解。比如有的新手会认为：“使用 C++的任何特性都没有成本”。那显然是大错特错的，比如使用模版就会导致编译时间变慢的编译期成本，而且我花了 21 天时间精通 C++的时间成本也是成本啊（狗头）。有些经验的 C++程序员会解释为”使用 C++的任何特性都没有**运行时**成本“，这也是对 C++最常见的误解。C++的创始人 Bjarne Stroustrup 是这样解释“零成本抽象“的：

1. 你不会为任何你没有使用的特性付出任何成本。
2. 对于你使用的特性，你只会付出最小运行时成本。

简单来说，就是 C++不会背着你偷偷干坏事（比如垃圾回收），而你指定 C++干的活，C++会尽量在编译期做，保证在运行期只会做“最少”的工作。连小学生都应该知道，“最少”并不等于“零”，所以“零成本抽象”其实是一种谎言，Google 的 C++负责人 Chandler Carruth 就曾经在 CppCon 2019 说过：C++根本不存在”零成本抽象“。

显然，C++的很多特性是有性能成本的，而且，这些成本往往出现在你“没有写”的代码里，即 C++帮你添加的隐形代码。作为 C++工程师，我们就必须了解每个特性会带来的性能损耗，在做代码设计时尽量选择损耗小的特性。

而下文介绍的很多坑点，C++语言服务器 clangd 可以帮你实时检测出来并自动修复。

（一）虚函数

老生常谈的性能损耗，这里只介绍一下虚函数调用带来的成本：

1. 会多一次寻址操作，去虚函数表查询函数的地址。
2. 可能会破坏 cpu 流水线，因为虚函数调用是一次间接调用，需要进行分支预测。
3. 阻碍了编译器内联，大部分情况下，虚函数是无法被内联的（与前两条相比，无法内联才是虚函数性能损耗的主要来源）。

然而在实际生产环境中，可能很多的运行时多态是无法避免的，毕竟这是 OOP 的基础特性，因此对于虚函数我们也只能了解背后的成本而已。某些情况下我们可以使用编译期多态来替代虚函数，比如 CRTP(Curiously Recurring Template Pattern)、Tempated Visitor Pattern、Policy Based Design 等等，我的下一篇文章《C++独有的设计模式》中会介绍这些技巧，敬请期待。

（二）隐形的拷贝

也是一个老生常谈的性能损耗，这里主要介绍几个容易被疏忽的场景：

Member Initialization 构造函数

```
class C {
 public:
  C(A a, B b): a_(a), b_(b){}
private:
  A a_;
  B b_;
}

int main() {
  A a;
  B b;
C c(a, b);
}
```

如果 A、B 是非平凡的类，会各被复制两次，在传入构造函数时一次，在构造时一次。C 的构造函数应当改为：

```
  C(A a, B b): a_(std::move(a)), b_(std::move(b)){}
```

这种写法是 clang-tidy 推荐的https://clang.llvm.org/extra/clang-tidy/checks/modernize/pass-by-value.html#pass-by-value-in-constructors ，相比传 const 引用进来，如果外面也是传右值，则完全没有拷贝。

For 循环

```
std::vector<std::string> vec;
for(std::string s: vec){
  // ...
}
```

这里每个 string 会被复制一次，改为 `for(const std::string& s: vec)` 即可.

Lambda 捕获

```
A a;
auto f = [a]{};
```

lambda 函数在值捕获时会将被捕获的对象拷贝一次，可以根据需求考虑使用引用捕获 `auto f = [&a]{};` 或者用 std::move 捕获初始化 `auto f = [a = std::move(a)]{};` （仅限 C++14 以后）。

隐式类型转换

```
std::unordered_map<int, std::string> map;
for(const std::pair<int, std::string>& p: map){
  // ...
}
```

这是一个很容易被忽视的坑点，这段代码用了 const 引用，但是因为类型错了，所以还是会发生拷贝，因为 unordered_map element 的类型是 `std::pair<const int, std::string>` ，所以在遍历时，推荐使用 const auto&，对于 map 类型，也可以使用结构化绑定。

（三）隐形的析构

在 C++代码中，我们几乎不会主动去调用类的析构函数，都是靠实例离开作用域后自动析构。而“隐形”的析构调用，也会导致我们的程序运行变慢：

复杂类型的析构

我们的业务代码中有这样一种接口

```
int Process(const Req& req, Resp* resp) {
  Context ctx = BuildContext(req); // 非常复杂的类型
```

```
  int ret = Compute(ctx, req, resp); // 主要的业务逻辑
  PrintTime();
  return ret;
}

int Api(const Req& req, Resp* resp) {
  int ret = Process(req, resp);
  PrintTime();
}
```

在日志中，Process 函数内打印的时间和 PrintTime 打印的时间竟然差了 20 毫秒，而我们当时接口的总耗时也不过几十毫秒，我当时百思不得其解，还是靠我老板 tomtang 一语道破先机，原来是析构 Context 足足花了 20ms。后面我们实现了 Context 的**池化**，直接将接口耗时降了 20%。

平凡析构类型

如何定义类的析构函数也大有讲究，看下下面这段代码：

```
class A {
 public:
  int i;
  int j;
  ~A() {};
};

A get() {
  return A{41, 42};
}
```

`get` 函数对应的汇编代码是：

```
get():                                # @get()
        movq    %rdi, %rax
        movabsq $180388626473, %rcx             # imm = 0x2A00000029
        movq    %rcx, (%rdi)
        retq
```

而如果我能把析构函数改一下：

```
class A {
 public:
  int i;
  int j;
  ~A() = default; // 注意这里
};
```

```
A get() {
  return A{41, 42};
}
```

对应的汇编代码则变成了：

```
get():                                # @get()
        movabsq $180388626473, %rax             # imm = 0x2A00000029
        retq
```

前者多了两次赋值，也多用了两个寄存器，原因是前者给类定义了一个自定义的析构函数（虽然啥也不干），会导致类为不可平凡析构类型(`std::is_trivially_destructible`)和不可平凡复制类型（`std::is_trivially_copyable`），根据 C++的函数调用 ABI 规范，不能被直接放在返回的寄存器中（%rax），只能间接赋值。除此之外，不可平凡复制类型也不能作为编译器常量进行编译器运算。所以，如果你的类是平凡的（只有数值和数字，不涉及堆内存分配），千万不要随手加上析构函数！

关于非平凡析构类型造成的性能损耗，后文还会多次提到。

（四）滥用 std::shared_ptr

C++核心指南是这样推荐智能指针的用法的：

1. 用 `std::unique_ptr` 或 `std::shared_ptr` 表达资源的所有权。
2. 不涉及所有权时，用裸指针。
3. 尽量使用 `std::unique_ptr`，只有当资源需要被共享所有权时，再用 `std::shared_ptr`。

但是在实际代码中，用 `std::shared_ptr` 的场景大概就是以下几种：

1. 小部分是因为代码作者是写 python 或者 java 来的，不会写没有 gc 的代码（比如 apache arrow 项目，所有数据全用 `std::shared_ptr`，像是被 apache 的 Java 环境给荼毒了）。
2. 绝大部分是因为代码作者是会写 C++的，但是太懒了，不想梳理内存资源模型。不得不说，`std::shared_ptr` 确实是懒人的福音，既保证了资源的安全，又不用梳理资源的所有权模型。
3. 很小一部分是因为确实需要使用 `std::shared_ptr` 的场景（不到 10%）。我能想到的必须用 std::shared_ptr 的场景有：异步析构，缓存。除此之外想不出任何必须的场景，欢迎小伙伴们在评论区补充。

实际上，`std::shared_ptr` 的构造、复制和析构都是非常重的操作，因为涉及到原子操作，`std::shared_ptr` 是要比裸指针和 `std::unique_ptr` 慢 10%～ 20%的。即使用了 `std::shared_ptr` 也要使用 std::move 和引用等等，尽量避免拷贝。

`std::shared_ptr` 还有个陷阱是一定要使用 `std::make_shared<T>()` 而不是 `std::shared_ptr<T>(new T)` 来构造，因为后者会分配两次内存，且原子计数和数据本身的内存是不挨着的，不利于 cpu 缓存。

（五）类型擦除：std::function 和 std::any

std::function，顾名思义，可以封装任何可被调用的对象，包括常规函数、类的成员函数、有 operator()定义的类、lambda 函数等等，当我们需要存储函数时 std::function 非常好用，但是 std::function 是有成本的：

1. std::function 要占用 32 个字节，而函数指针只需要 8 个字节
2. std::function 本质上是一个虚函数调用，因此虚函数的问题 std::function 都有，比如无法内联
3. std::function 可能涉及堆内存分配，比如 lambda 捕获了大量值时，用 std::function 封装会需要在堆上分配内存

因此我们只应在必须时才使用 std::function，比如需要存储一个不确定类型的函数。而在只需要多态调用的，完全可以用模版静态派发：

```
template <typename Func>
void Run(Func&& f){
  f();
}
```

std::any 同理，用类型擦除的机制可以存储任何类型，但是也不推荐使用。

（六）std::variant 和 std::optional

我在我的另一篇文章《C++17 在业务代码中最好用的十个特性》大肆吹捧了一波 std::variant 和 std::optional，但是说实话，C++的实现还是有些性能开销的，这里以 std::optional 为例介绍：

1. 必须的多余内存开销：简单来说，std::optional有两个成员变量，类型分别为 bool 和 T，由于内存对齐的原因，sizeof(std::optional)会是 sizeof(T)+alignof(T)，对于简单的类型，sizeof(std::optional)甚至会是 sizeof(T)的两倍。相比之下，rust 语言的 option 实现则有 null pointer optimization，即如果一个类的合法内存表示一定不会全部字节为零，比如 std::reference_wrapper，那就可以零开销地表示 std::optional，而 C++由于需要兼容 C 的内存对齐，不可能实现这项优化
2. c++标准要求如果 T 是可平凡析构的（见上文析构的部分），std::optional也必须是平凡析构的，但是 gcc 在 8.0.0 之前的实现是有 bug 的，所有 std::optional都被设置为了非平凡类型，所以用 std::optional 作为工厂函数的返回值是由额外性能开销的。
3. 对 NRVO（返回值优化）不友好，见下文 NRVO 的部分。

（七）std::async

std::async 是一个很好用的异步执行抽象，但是在使用的时候可能一不小心，你的代码就变成了同步调用：

不指定 policy

std::async 的接口是：

```
template< class Function, class... Args >
std::future<std::invoke_result_t<std::decay_t<Function>, std::decay_t<Args>...>>
    async( Function&& f, Args&&... args );

template< class Function, class... Args >
std::future<std::invoke_result_t<std::decay_t<Function>, std::decay_t<Args>...>>
    async( std::launch policy, Function&& f, Args&&... args );
```

其中 std::launch 类型包括两种：**std::launch::async**异步执行和**std::launch::deferred**懒惰执行，如果你使用第一种接口不指定 policy，那么编译器可能会自己帮你选择懒惰执行，也就是在调用 future.get()的时候才同步执行。

不保存返回值

这是 c++的 std::async 的一个大坑点，非常容易踩坑，比如这段代码：

```
void func1() {
// ...
}

void func2() {
// ...
}

int main() {
std::async(std::launch::async, func1);
std::async(std::launch::async, func2);
}
```

在这段代码里，func1 和 func2 其实是串行的！因为 std::async 会返回一个 std::future，而这个 std::future 在析构时，会同步等待函数返回结果才析构结束。这也是上文“隐形的析构”的另外一种表现。正确的代码应当长这样：

```
auto future1 = std::async(std::launch::async, func1);
auto future2 = std::async(std::launch::async, func2);
```

更奇葩的是，只有 std::async 返回的 std::future 在析构时会同步等待，std::packaged_task，std::promise 构造的 std::future 都不会同步等待，实在是让人无力吐槽。

关于 std::async 等等 C++多线程工具，在我之后的文章《现代 C++并发编程指南》会介绍，敬请期待。

与编译器作对

众所周知，现代编译器是非常强大的。毛主席曾经说过：要团结一切可以团结的力量。面对如此强大的编译器，我们应该争取做编译器的朋友，而不是与编译器为敌。做编译器的朋友，就是要充分利用编译器的优化。而很多优化是有条件的，因此我们要争取写出优化友好的代码，把剩下的工作交给编译器，而不是自己胡搞蛮搞。

（八）返回值优化 NRVO（Named Return Value Optimization）

当一个函数的返回值是当前函数内的一个局部变量，且该局部变量的类型和返回值一致时，编译器会将该变量直接在函数的返回值接收处构造，不会发生拷贝和移动，比如：

```
#include <iostream>

struct Noisy {
  Noisy() { std::cout << "constructed at " << this << '\n'; }
  Noisy(const Noisy&) { std::cout << "copy-constructed\n"; }
  Noisy(Noisy&&) { std::cout << "move-constructed\n"; }
  ~Noisy() { std::cout << "destructed at " << this << '\n'; }
};

Noisy f() {
  Noisy v = Noisy();
return v;
}
void g(Noisy arg) { std::cout << "&arg = " << &arg << '\n'; }

int main() {
  Noisy v = f();
  g(f());
}
```

这段代码中，函数 f()满足 NRVO 的条件，所以 Noisy 既不会拷贝，也不会 move，只会被构造和析构两次，程序的输出：

```
constructed at 0x7fff880300ae
constructed at 0x7fff880300af
&arg = 0x7fff880300af
destructed at 0x7fff880300af
destructed at 0x7fff880300ae
```

滥用 std::move

自从 C++11 加入 std::move 语义之后，有些“自以为是”的程序员会到处添加 move。在这些情况下，std::move 是根本没用的：

1. 被 move 的对象是平凡类型。

2. 被 move 的对象是一个常量引用。

而在某些情况下，move 反而会导致负优化，比如阻碍了 NRVO：

```
Noisy f() {
  Noisy v = Noisy();
  return std::move(v);
}
```

还是上面的代码，只不过返回值被改成 move 走，结果就多了两次 move 构造和两次析构，反而得不偿失：

```
constructed at 0x7ffc54006cdf
move-constructed
destructed at 0x7ffc54006cdf
constructed at 0x7ffc54006cdf
move-constructed
destructed at 0x7ffc54006cdf
&arg = 0x7ffc54006d0f
destructed at 0x7ffc54006d0f
destructed at 0x7ffc54006d0e
```

工厂返回 std::optional

同样的，使用 std::optional 也可能会阻碍 NRVO 优化：

```
std::optional<Noisy> f() {
  Noisy v = Noisy();
  return v;
}
```

因为返回值类型不对应，因此应当改为

```
std::optional<Noisy> f() {
  std::optional<Noisy> v;
  v = Noisy();
  return v;
}
```

为了性能牺牲了部分可读性。

（九）尾递归优化

尾递归优化是函数式语言常用的一种优化，如果某个函数的最后一步操作是调用自身，那么编译器完全可以不用调用的指令(call)，而是用跳转(jmp)回当前函数的开头，省略了新开调用栈的开销。然而由于 C++的各种隐形操作，尾递归优化不是那么好实现。我曾经在知乎上看到这样一个问题：**https://www.zhihu.com/question/552352098**。题主的函数长这样：

```
unsigned btd_tail(std::string input, int v) {
    if (input.empty()) {
        return v;
    } else {
        v = v * 2 + (input.front() - '0');
        return btd_tail(input.substr(1), v);
    }
}
```

直接 return 自身的调用，如果在函数式语言就是一个标准的尾递归，然而，实际执行的代码是：

```
unsigned btd_tail(std::string input, int v) {
    if (input.empty()) {
        return v;
    } else {
        v = v * 2 + (input.front() - '0');
        auto temp = btd_tail(input.substr(1), v);
        input.~string(); // 注意这里
        return temp;
    }
}
```

由于在 return 前 C++有隐形的析构操作，所以这段代码并不是尾递归。而需要析构的本质原因是 std::string 不是可平凡析构的对象，解决办法也很简单，换成 std::string_view 就好了

```
unsigned btd_tail(std::string_view input, int v) {
    if (input.empty()) {
        return v;
    } else {
        v = v * 2 + (input.front() - '0');
        return btd_tail(input.substr(1), v);
    }
}
```

std::string_view 是可平凡析构的，所以编译器根本不需要调用析构函数，这也是上文推荐尽量选用可平凡析构对象的另一个理由。我的下一篇文章《C++函数式编程指南》会介绍C++函数式编程，敬请期待。

（十）自动向量化优化

现代 CPU 大部分都支持一些向量化指令集如 SSE、AVX 等，向量化指的是 SIMD 操作，即一个指令，多条数据。在某些条件下，编译器会自动将循环优化为向量化操作：

1. 循环内部访问的是连续内存
2. 循环内部没有函数调用，没有 if 分支
3. 循环之间没有依赖

举个例子，下方的代码非常的向量化不友好：

```
enum Type { kAdd, kMul };
int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

std::vector<int> func(std::vector<int> a, std::vector<int> b, Type t) {
std::vector<int> c(a.size());
for (int i = 0; i < a.size(); ++i) {
    if (t == kAdd) {
      c[i] = add(a[i], b[i]);
    } else {
      c[i] = mul(a[i], b[i]);
    }
  }
return c;
}
```

既有 if，又有函数调用，而如果我们通过模版 if 和内联函数，这两条都可以规避：

```
enum Type { kAdd, kMul };
inline __attribute__((always_inline)) int add(int a, int b) { return a + b; }
inline __attribute__((always_inline)) int mul(int a, int b) { return a * b; }

template <Type t>
std::vector<int> func(std::vector<int> a, std::vector<int> b) {
std::vector<int> c(a.size());
for (int i = 0; i < a.size(); ++i) {
    if constexpr (t == kAdd) {
      c[i] = add(a[i], b[i]);
    } else {
      c[i] = mul(a[i], b[i]);
    }
```

```
  }
return c;
}
```

这样就变成了向量化友好的代码。我们团队正在基于 apache arrow 做一些向量化计算的工作，后续也会有文章分享关于向量化优化的详细介绍。

下篇介绍10个C++在业务代码中最好用的特性。

> 自从步入现代 C++时代开始，C++语言标准形成了三年一个版本的惯例：C++11 标志着现代 C++的开端，C++14 在 11 的基础上查缺补漏，并未加入许多新特性，而 C++17 作为 C++11 后的第一个大版本，标志着现代 C++逐渐走向成熟。WXG 编译器升级到 gcc7.5 已有一段时间，笔者所在项目组也已经将全部代码升级到 C++17。在使用了 C++17 一年多之后，笔者总结了 C++17 在业务代码中最好用的十个特性。

注 1：本文只包含 wxg 的 gcc7.5 支持的特性，**Execution Policy**, **File System**等暂不支持的特性不包含在内。

注 2：本文只包含应用于业务逻辑的特性，**Fold Expression**, **Mathematical Special Functions**等适用于元编程和科学计算的特性并不包含。

笔者将这些特性大体上分为三类：语法糖、性能提升和类型系统。

语法糖

这里所说的语法糖，并不是严格意义上编程语言级别的语法糖，还包括一些能让代码更简洁更具有可读性的函数和库：

结构化绑定

c++17 最便利的语法糖当属结构化绑定。结构化绑定是指将 array、tuple 或 struct 的成员绑定到一组变量*上的语法，最常用的场景是在遍历 map/unordered_map 时不用再声明一个中间变量了：

```
// pre c++17
for(constauto& kv: map){
constauto& key = kv.first;
constauto& value = kv.second;
// ...
```

```
}

// c++17
for(constauto& [key, value]: map){
// ...
}
```

*: 严格来说，结构化绑定的结果并不是变量，c++标准称之为名字/别名，这也导致它们不允许被 lambda 捕获，但是 gcc 并没有遵循 c++标准，所以以下代码在 gcc 可以编译，clang 则编译不过

```
for(const auto& [key, value]: map){
    [&key, &value]{
        std::cout << key << ": " << value << std::endl;
    }();
}
```

在 clang 环境下，可以在 lambda 表达式捕获时显式引入一个引用变量通过编译

```
for(const auto& [key, value]: map){
    [&key = key, &value = value]{
        std::cout << key << ": " << value << std::endl;
    }();
}
```

另外这条限制在 c++20 中已经被删除，所以在 c++20 标准中 gcc 和 clang 都可以捕获结构化绑定的对象了。

std::tuple 的隐式推导

在 c++17 以前，构造 `std::pair/std::tuple` 时必须指定数据类型或使用 `std::make_pair/std::make_tuple` 函数，c++17 为 `std::pair/std::tuple` 新增了推导规则，可以不再显示指定类型。

```
// pre c++17
std::pair<int, std::string> p1{3.14, "pi"s};
auto p1 = std::make_pair(3.14, "pi"s);

// c++17
std::pair p3{3.14, "pi"s};
```

if constexpr

if constexpr 语句是编译期的 if 判断语句，在 C++17 以前做编译期的条件判断往往通过复杂**SFINAE**机制或模版重载实现，甚至嫌麻烦的时候直接放到运行时用 if 判断，造成性能

损耗，if constexpr 大大缓解了这个问题。比如我想实现一个函数将不同类型的输入转化为字符串，在 c++17 之前需要写三个函数去实现，而 c++17 只需要一个函数。

```
// pre c++17
template <typename T>
std::string convert(T input){
    return std::to_string(input);
}

// const char*和string进行特殊处理
std::string convert(const char* input){
    return input;
}
std::string convert(std::string input){
    return input;
}
```

```
// c++17
template <typename T>
std::string convert(T input) {
    if constexpr (std::is_same_v<T, const char*> ||
                  std::is_same_v<T, std::string>) {
        return input;
    } else {
        return std::to_string(input);
    }
}
```

if 初始化语句

c++17 支持在 if 的判断语句之前增加一个初始化语句，将仅用于 if 语句内部的变量声明在 if 内，有助于提升代码的可读性。且对于 lock/iterator 等涉及并发/RAII 的类型更容易保证程序的正确性。

```
// c++ 17
std::map<int, std::string> m;
std::mutex mx;
externbool shared_flag; // guarded by mx

int demo()
{
    if (auto it = m.find(10); it != m.end()) { return it->second.size(); }
    if (char buf[10]; std::fgets(buf, 10, stdin)) { m[0] += buf; }
    if (std::lock_guard lock(mx); shared_flag) { unsafe_ping(); shared_flag = false; }
```

```
    if (int s; int count = ReadBytesWithSignal(&s)) { publish(count); raise(s); }
    if (constauto keywords = {"if", "for", "while"};
        std::ranges::any_of(keywords, [&tok](constchar* kw) { return tok == kw; }))
    {
        std::cerr << "Token must not be a keyword\n";
    }
}
```

性能提升

std::shared_mutex

shared_mutex 是 c++的原生读写锁实现，有共享和独占两种锁模式，适用于并发高的读场景下，通过 reader 之前共享锁来提升性能。在 c++17 之前，只能自己通过独占锁和条件变量自己实现读写锁或使用 c++14 加入的性能较差的 std::shared_timed_mutex 。以下是通过 shared_mutex 实现的线程安全计数器：

```
// c++17
class ThreadSafeCounter {
public:
  ThreadSafeCounter() = default;

// Multiple threads/readers can read the counter's value at the same time.
unsigned int get() const {
    std::shared_lock lock(mutex_);
    return value_;
  }

// Only one thread/writer can increment/write the counter's value.
unsigned int increment() {
    std::unique_lock lock(mutex_);
    return ++value_;
  }

// Only one thread/writer can reset/write the counter's value.
void reset() {
    std::unique_lock lock(mutex_);
    value_ = 0;
  }

private:
mutablestd::shared_mutex mutex_;
unsignedint value_ = 0;
};
```

std::string_view

std::string_view 顾名思义是字符串的“视图”，类成员变量包含两个部分：字符串指针和字符串长度，std::string_view 涵盖了 std::string 的所有只读接口。std::string_view 对字符串不具有所有权，且兼容 std::string 和 const char*两种类型。

c++17 之前，我们处理只读字符串往往使用 const std::string& ， std::string 有两点性能优势:

1. 兼容两种字符串类型，减少类型转换和内存分配。如果传入的是明文字符串 const char *，const std::string& 需要进行一次内存分配，将字符串拷贝到堆上，而 std::string_view 则可以避免。
2. 在处理子串时， std::string::substr 也需要进行拷贝和分配内存，而 std::string_view::substr 则不需要，在处理大文件解析时，性能优势非常明显。

```
// from https://stackoverflow.com/a/40129046
// author: Pavel Davydov

// string_view的remove_prefix比const std::string&的快了15倍
string remove_prefix(const string &str) {
return str.substr(3);
}
string_view remove_prefix(string_view str) {
  str.remove_prefix(3);
return str;
}

static void BM_remove_prefix_string(benchmark::State& state) {
std::string example{"asfaghdfgsghasfasg3423rfgasdg"};
while (state.KeepRunning()) {
    auto res = remove_prefix(example);
    // auto res = remove_prefix(string_view(example)); for string_view
    if (res != "aghdfgsghasfasg3423rfgasdg") {
      throwstd::runtime_error("bad op");
    }
  }
}
```

std::map/unordered_map try_emplace

在向 std::map/unordered_map 中插入元素时，我们往往使用 emplace ， emplace 的操作是如果元素 key 不存在，则插入该元素，否则不插入。但是在元素已存在时， emplace 仍会构造一次待插入的元素，在判断不需要插入后，立即将该元素析构，因此进行了一次多余构造和析构操作。c++17 加入了 try_emplace ，避免了这个问题。同时 try_emplace 在参数列表中将 key 和 value 分开，因此进行原地构造的语法比 emplace 更加简洁

```
std::map<std::string, std::string> m;
// emplace的原地构造需要使用std::piecewise_construct，因为是直接插入std::pair<key, value>
```

```
m.emplace(std::piecewise_construct,
          std::forward_as_tuple("c"),
          std::forward_as_tuple(10, 'c'));

// try_emplace可以直接原地构造，因为参数列表中key和value是分开的
m.try_emplace("c", 10, 'c')
```

同时，c++17 还给 std::map/unordered_map 加入了 insert_or_assign 函数，可以更方便地实现插入或修改语义。

类型系统

c++17 进一步完备了 c++的类型系统，终于加入了众望所归的类型擦除容器(**Type Erasure**)和代数数据类型(**Algebraic Data Type**)

std::any

std::any 是一个可以存储任何可拷贝类型的容器，C 语言中通常使用 void* 实现类似的功能，与 void* 相比， std::any 具有两点优势：

1. std::any 更安全：在类型 T 被转换成 void* 时，T 的类型信息就已经丢失了，在转换回具体类型时程序无法判断当前的 void* 的类型是否真的是 T，容易带来安全隐患。而 std::any 会存储类型信息， std::any_cast 是一个安全的类型转换。
2. std::any 管理了对象的生命周期，在 std::any 析构时，会将存储的对象析构，而 void* 则需要手动管理内存。

std::any 应当很少是程序员的第一选择，在已知类型的情况下， std::optional ， std::variant 和继承都是比它更高效、更合理的选择。只有当对类型完全未知的情况下，才应当使用 std::any ，比如动态类型文本的解析或者业务逻辑的中间层信息传递。

std::optional

std::optional<T> 代表一个可能存在的 T 值，对应 Haskell 中的 Maybe 和 Rust/OCaml 中的 option ，实际上是一种**Sum Type**。常用于可能失败的函数的返回值中，比如工厂函数。在 C++17 之前，往往使用 T* 作为返回值，如果为 nullptr 则代表函数失败，否则 T* 指向了真正的返回值。但是这种写法模糊了所有权，函数的调用方无法确定是否应该接管 T* 的内存管理，而且 T* 可能为空的假设，如果忘记检查则会有 SegFault 的风险。

```
// pre c++17
ReturnType* func(const std::string& in) {
    ReturnType* ret = new ReturnType;
    if (in.size() == 0)
        returnnullptr;
    // ...
    return ret;
```

```
}

// c++17 更安全和直观
std::optional<ReturnType> func(const string& in) {
    ReturnType ret;
    if (in.size() == 0)
        return nullopt;
    // ...
    return ret;
}
```

std::variant

`std::variant<T, U, ...>` 代表一个多类型的容器，容器中的值是制定类型的一种，是通用的 Sum Type，对应 Rust 的 `enum`。是一种类型安全的 `union`，所以也叫做 `tagged union`。与 `union` 相比有两点优势：

1. 可以存储复杂类型，而 union 只能直接存储基础的 POD 类型，对于如 `std::vector` 和 `std::string` 就等复杂类型则需要用户手动管理内存。
2. 类型安全，variant 存储了内部的类型信息，所以可以进行安全的类型转换，c++17 之前往往通过 `union` + `enum` 来实现相同功能。

通过使用 `std::variant<T, Err>`，用户可以实现类似 Rust 的 `std::result`，即在函数执行成功时返回结果，在失败时返回错误信息，上文的例子则可以改成：

```
std::variant<ReturnType, Err> func(const string& in) {
    ReturnType ret;
    if (in.size() == 0)
        return Err{"input is empty"};
    // ...
    return {ret};
}
```

需要注意的是，c++17 只提供了一个库级别的 variant 实现，没有对应的**模式匹配(Pattern Matching)**机制，而最接近的 `std::visit` 又缺少编译器的优化支持，所以在 c++17 中 `std::variant` 并不好用，跟 Rust 和函数式语言中出神入化的 Sum Type 还相去甚远，但是已经有许多围绕 `std::variant` 的提案被提交给 c++委员会探讨，包括模式匹配，`std::expected` 等等。

总结一下，c++17 新增的三种类型给 c++带来了更现代更安全的类型系统，它们对应的使用场景是：

- `std::any` 适用于之前使用 `void*` 作为通用类型的场景。
- `std::optional` 适用于之前使用 `nullptr` 代表失败状态的场景。
- `std::variant` 适用于之前使用 `union` 的场景。

总结

以上是笔者在生产环境中最常用的 c++17 特性，除了本文描述的十个特性外，c++17 还添加了如**lambda 值捕获*this**, **钳夹函数 std::clamp()**, **强制检查返回值[[nodiscard]]**等非常易用的特性，本文篇幅有限不做赘述，欢迎有兴趣的读者自行探索。



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
690篇原创内容
公众号



阅读 5736

留言

写留言

还没有留言，来写首评